BonusMatch3Demo
The Sample Project Mini-Cookbook

This FREE booklet is designed to accompany the sample project `BonusMatch3Demo` from the book Practical C++ Game Programming with Data Structures and Algorithms. It provides additional insights into the code logic, algorithms, and problem-solving approaches used in the implementation of this sample.

In this mini booklet, you'll find how the sample project works and the key ideas behind its design.

Match-3 puzzles—such as *Candy Crush* or *Bejeweled*—may appear deceptively simple on the surface. Yet behind every falling candy lies a precise combination of bit manipulation, heuristic search, flood-fill algorithms, and finite state machines.

This project demonstrates how to build a complete match-3 system. We'll walk through the core algorithms and data structures that make the game efficient, deterministic, and easy to extend.

Before we begin

You'll need the following background knowledge from the book:

- Chapter 2: Common data structures
- Chapter 4: Real-time 2D graphics manipulation
- Chapter 7: Particle effects
- Chapter 8: Animation concepts
- Chapter 10: Finite State Machines

Some basic understanding of raylib is a plus. It's the best time to try out this sample after you finish reading Chapter 10. This booklet will not repeatedly cover anything you need to know already in the book.

Getting project up and running

First, make sure you have pulled the latest updates from GitHub (https://github.com/PacktPublishing/Practical-C-Game-Programming-with-Data-Structures-and-Algorithms/tree/main).

Open the Visual Studio solution file included with this book in Visual Studio Community 2022, select the `BonusMatch3Demo` project as start-up project, rebuild it, and run. You should see the similar scene like *Figure 1*.

Figure 1 – Running the sample project BonusMatch3Demo

There are five different chip colors on the board: red, orange, blue, purple, and green. Use the left mouse button to select a chip, then drag it to swap with a neighboring chip to form a line of at least **three matching colors**. Each match removes the matched chips, and new chips fall in to refill the board. This can trigger additional matches, creating a **chain reaction** that continues until no more matches are possible.

When you create a match of four chips in a row, a special chip called a Cross Bomb appears like *Figure 2*. Clicking it causes an explosion that clears all adjacent chips—up, down, left, and right—regardless of their color.

Figure 2 – Match 4 chips in a row creates a special Cross Bomb

When you create a match of five chips in a row, or six chips forming a T-shape, a special chip called a Color Bomb appears like *Figure 3*.

Figure 3 – Color Bomb is created when you get a match of more than 4 same-color chips

Clicking the Color Bomb destroys all chips of the same color on the board, no matter where they are located.

Occasionally, you may reach a state where no possible match of three same-colored chips can be made—a dead end. While rare, it can happen. When the game detects that no matches are available, it automatically triggers a shuffle operation, rearranging all chips on the board until at least one valid match is found.

If you remain idle for more than 20 seconds, the game will automatically search for potential moves and highlight the best chip you can swap to continue playing like *Figure 4*.

Figure 4 – A hint is shown after idle for 20 seconds

Main game state changes in Match3GameSession

The majority of gameplay logic is managed within the `Match3GameSession` class. The entire match-3 session operates as a simple finite state machine, cycling through the following sequence:

Idle → Swapping → Clearing → FallingCollapse → FallingRefill → Idle

Idle State

- The `Idle` state waits for player input.
- The board is stable (no tiles are falling or being cleared).
- The player can click and drag to select a chip to swap.
- The hint system may highlight a suggested move if the player remains inactive.
- Transitions:
  → `Swapping` when the player initiates a valid swap.

Swapping State

- This state handles the swap animation and match validation.
- The two selected tiles exchange positions with a short easing animation.
- After the animation, the game checks whether the swap created any matches.
- Transitions:
  → `Clearing` if the swap resulted in one or more matches.
  → `Reverting` if no matches were made (invalid move).

Reverting State

- Handles undoing invalid swaps.
- The swapped tiles animate back to their original positions.

- The board state remains unchanged (no tiles are cleared).
- Transitions:
  → `Idle` once the revert animation finishes.

Clearing State

- Handles the removal of matched tiles and any special effects.
- All matched tiles are marked and cleared.
- Special tiles (e.g., Cross Bomb or Color Bomb) are spawned based on the match size.
- Particle bursts, screen shakes, and flashes may trigger depending on the match type.
- Transitions:
  → `FallingCollapse` once all clearing animations are completed.

FallingCollapse State

- Simulates gravity after clearing.
- Tiles above empty cells fall downward using a column compaction algorithm.
- Vertical movement (offY offsets) is animated to show tiles dropping.
- Transitions:
  → `FallingRefill` after all movement is finished.

FallingRefill State

- Refills newly empty tiles.
- New random tiles appear at the top and fall into empty spaces.
- The game checks whether the refill caused any new automatic matches.

- Transitions:
  → `Clearing` if new matches were created (cascading reaction).
  → `Shuffling` if no legal moves remain after refill.
  → `Idle` if the board is stable and playable again.

Shuffling State

- Recovers from a “no move available” situation.
- Randomly rearranges all tiles while ensuring:
- No pre-existing matches exist after the shuffle.
- At least one valid move is available.
- Often accompanied by a short shake or animation to indicate the reshuffle.
- Transitions:
  → `FallingRefill` to settle the new tiles before returning to normal gameplay flow.

Each state encapsulates a phase of gameplay. Transitions are driven by timers (`swap.t`, `clearAnimTime`) and logical checks (function `AnyEmpty()`).

This design keeps `Update()` funciton in `Match3GameSession` class linear and readable, avoiding deeply nested conditions.

The match board

The board is a fixed 8×8 grid (defined in `Match3Board.h`):

```
int  cell[ROWS][COLS];     // -1 empty else [0..CHIP_TYPES-1]
int  special[ROWS][COLS];  // SpecialType (SP_CrossBomb, SP_ColorBomb)
float offX[ROWS][COLS];   // pixel X-offset for anim (e.g., during swap)
float offY[ROWS][COLS];   // pixel Y-offset for anim (e.g., during fall)
float highlight[ROWS][COLS];  // transient glow/selection strength [0..1]
```

This compact 2-D array allows direct indexing to speed up most operations.

But many queries—"which chips form a triple?" or "which cells are empty?"—are all *parallel* and *bit-wise* in nature.

For that reason, the engine introduces `Bitboard`.

Using bitboard to speed up operations

Each color has its own `uint64_t` bitboard. Bit `i` represents whether the tile at index `i = row × 8 + col` holds that color of chip. (in `Bitboard.h`)

```
std::array<uint64_t, CHIP_TYPES> bb;
```

So if you look at the color chips on the match board in *Figure 5*:

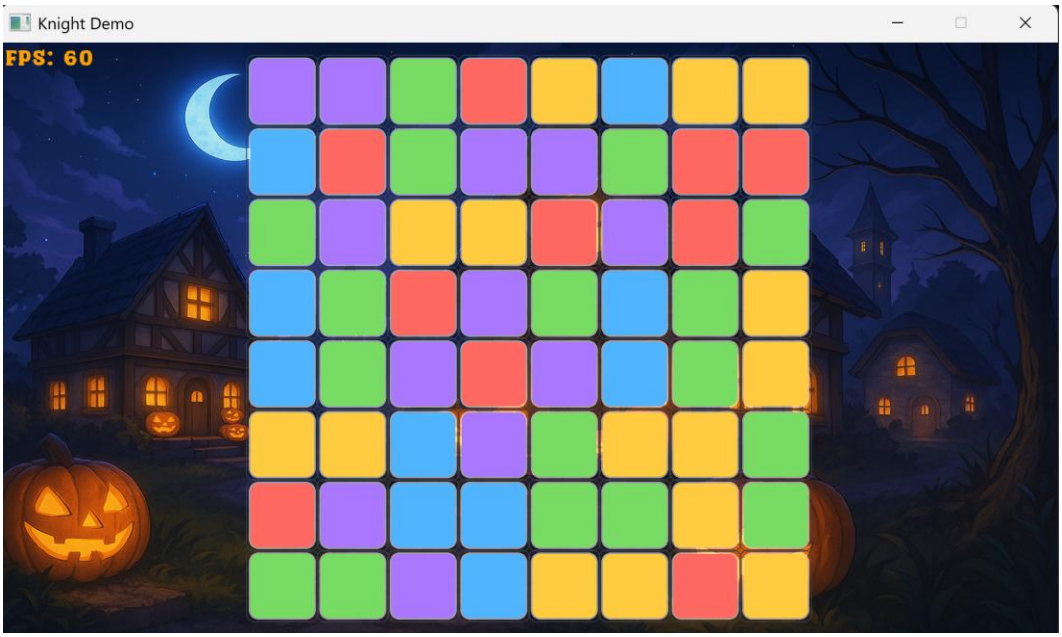

Figure 5 – The color chip on the match board

In the bitboard of blue color, it will look like the following:

```
Row 1 - 0 0 0 0 0 1 0 0

Row 2 - 1 0 0 0 0 0 0 0
```

```
Row 3 - 0 0 0 0 0 0 0 0
Row 4 - 1 0 0 0 0 1 0 0
Row 5 - 1 0 0 0 0 1 0 0
Row 6 - 0 0 1 0 0 0 0 0
Row 7 - 0 0 1 1 0 0 0 0
Row 8 - 0 0 0 1 0 0 0 0
```

Detecting Matches with Bit-Shifts

Horizontal triples can be found by shifting the bitboard (`TriplesHExpanded` function):

```
uint64_t a = b & (b >> 1);      // pairs
uint64_t s = a & (b >> 2);      // triplets
```

Vertical triples use 8-bit strides (>> 8, >> 16).

The algorithm expands each detected triple to a full 3-cell mask and ORs all colors together.

One interesting observation is the bitboards implementation offer SIMD (Single Instruction/Multiple Data) -like speed without GPU compute, it runs like 64-bit parallelism.

This vectorized technique replaces $O(n^2)$ nested `for` loops with constant-time bit operations, which is a major speed-up.

Flood Fill for Connected Groups

To recognize larger connected clusters (for special bonuses), the sample code uses a **4-connected flood fill** implemented entirely in bit arithmetic:

```
static uint64_t FloodFill4(uint64_t groupMask, uint64_t seed) {
  uint64_t comp = 0, frontier = seed & groupMask;
  while (frontier) {
    comp |= frontier;
```

```
    uint64_t L = (frontier << 1) & NOT_COL0;
    uint64_t R = (frontier >> 1) & NOT_COL7;
    uint64_t U = (frontier << 8);
    uint64_t D = (frontier >> 8);
    frontier = (L | R | U | D) & groupMask & ~comp;
    }
  return comp;
}
```

Note the magic constants `NOT_COL0` and `NOT_COL7` are defined in `Bitboard.h`:

```
// Precomputed row/column masks for fast horizontal/vertical ops.
static inline uint64_t RowMask(int r) { return 0xFFULL << (r * 8); }
static const uint64_t NOT_COL0 = 0xfefefefefefefefeULL;
static const uint64_t NOT_COL7 = 0x7f7f7f7f7f7f7f7fULL;
```

This computes connected components without recursion or queues—O(k) over cluster size but with cache-friendly bit ops.

Simulating gravity

When chips disappear (being matched and eliminated), the remaining ones“fall” to fill the gaps.

Each column is processed independently using a simple linear scan algorithm:

1. Traverse from bottom to top of each column.
2. Maintain a `write pointer` (write = last empty row).
3. When a non-empty cell is found, move it to write, then decrement write.

This generates a list of (src, dst) moves:

```
// A single falling move: move from (srcR, c) to (dstR, c) within a
column.
struct FallPlanEntry {
    int srcR, dstR, c;
```

```
};
```

The collected plan is applied in bulk to both the color grid and the “special” grid, and offsets for animation are accumulated. This is an example of a **compaction algorithm**, like those used to remove holes in memory allocation management.

Refilling chips

After compaction, any cell with `cell[r][c] == -1` is filled with a random color chip from a `std::mt19937` RNG stream (defined in RandomGenerator.h).

Refilling is deterministic if seeded, which helps for advanced topic like debugging and replay systems for match three game simulation.

Detecting and Clearing Matches

Match Detection

The function `FindMatchesWithSpawns()` in `Match3GameSession` class marks all triples using bitboard masks and then groups them by color via the flood-fill.

Group size determines which special piece to spawn:

- 3 = normal clear
- 4 = Cross Bomb
- 5 + = Color Bomb

Spawn positions prefer the swap origin, ensuring predictable results.

Clearing matched chips

Clearing applies a boolean mask:

```
if (mask[r][c] && !isSpawn[r][c])
    cell[r][c] = -1;
```

The mask is combined across colors with OR.

Move Generation and Heuristics

To find hints or AI moves, the sample uses a **heuristic** move generator. The term *heuristic* refers to strategies designed to efficiently solve complex optimization problems by producing approximate solutions when exact methods are impractical.

The algorithm works as follows:

1. Enumerate all adjacent tile pairs *(A, B)* that could be swapped.
2. Temporarily swap each pair on a copy of the board.
3. Simulate the resulting cascades by repeatedly:
    - Finding matches.
    - Clearing → collapsing → refilling.
    - Counting the total number of tiles cleared.
4. Score each move based on the total number of tiles cleared.

The move with the highest score is selected as the suggested hint.

This method effectively performs a greedy local search using simulated rollouts. For small boards, this brute-force approach remains efficient, since each step relies on bitboard-based match detection to speed up evaluation.

Randomness and Determinism

The sample maintains four independent RNG streams:

```
struct RNGStreams {
    std::mt19937 board, particles, hint, shuffle;
};
```

Splitting them ensures that particle or UI randomness never affects the puzzle logic—a good software-engineering practice separating **game logic determinism** from **visual noise**.

Object Pooling in particle system

We introduced how 3D particle systems work in *Chapter 7*.
Using a full 3D billboard-based particle system here would be excessive, so instead, we implement a **simple 2D particle system** with optimized memory allocation through object pooling.

Rather than allocating new particles every frame, the system maintains a **particle pool** that uses a fixed-size vector and a **free-index stack** to recycle existing particles efficiently.

```
std::vector<Particle2D> pool;

std::vector<int> freeIdx;
```

When a particle dies, its index is pushed back onto the free list.This avoids dynamic memory churn and illustrates a classic object-pool pattern for real-time systems.

Conclusion

We used the `BonusMatch3Demo` project to demonstrate the algorithms and data structures behind a match-three game session.

For performance analysis:

- Match detection uses a `bitboard` representation, achieving O(1) complexity per color.
- The flood-fill algorithm operates at O(n), where *n* is the cluster size, which remains efficient in practice.
- The gravity simulation runs in O(rows × columns) per column scan.
- The heuristic move search algorithm performs at O(n × swap_candidates) but is limited to evaluating a maximum of 100 swaps for efficiency.

The separation between game simulation (handled in the `Update()` function) and visual rendering (handled in the `Draw()` function) follows the same `Component`-based design principles, even though the `Match3GameSession` class itself is not implemented as a Knight `Component`.

However, this is only the beginning of your journey into match-3 gameplay design. We'll return to our match-3 implementation sometime next year to explore advanced features, AI behaviors, and performance optimizations that take the system even further.

Challenging yourself

We kept a lot of configurable parameters inside `Match3GameConfig.h` so you have the freedom to tweak them all.

However, there is a catch, change the size of match board from 8x8 is NOT that simple.

Our current implementation is fixed to an 8x8 match board. What about to support a more flexible M x N match board? Apparently using a 64-bit long int is no longer an option. You might need to consider using `std::bitset` data structure for better flexibility.

Happy coding!